ml_edu.results.plot_model predictions

The `ml_edu.results.plot_model_predictions` function is a visualization utility from Google's Machine Learning Education package used in [Google Colab Exercises](#) to plot a trained model's predictions against actual data values for one or two input features. Google Colab +1

6 sites

linear_regression_taxi.ipynb - Google Colab
Code - Experiment 1. Hücreyi çalıştır (Ctrl+Enter). hücre bu oturumda yürütülmedi. [ ]. #@title Code - Experiment 1 #...
Google Colab

plot_predictions and sjPlot::plot_model for prediction...
I'm estimating a multilevel model with lme4. Previously, I'd visualize a conditional...
GitHub

Releases · google/ml-edu - GitHub
Make classification threshold optional so experiment settings can also be used with regression. Add method to...

Core Function Details

- **Purpose**: Visualizes how well your trained model's predictions fit the actual target values using the dataset.
- **Arguments**: Typically takes the trained experiment object (`experiment`), the dataset (`training_df` or test data), and the target label string.
- **Supported Inputs**: Handles models with single or double input features to render scatter plots or decision/prediction lines. Google Colab +1

Usage Example

python

~~~python
ml_edu.results.plot_model_predictions(experiment_1, training_df, 'FARE')
```

<FollowUp>
If you need help with **setting up the experiment parameters**, **interpr
</FollowUp>
~~~

Use code with caution.

It shows steps. what is it?

The "steps" you are seeing are the **classification decision boundary** being evaluated over discrete data ranges, or a visualization of a **decision tree / step-function model**.

Depending on the specific Google Colab exercise you are running, here is exactly what those steps mean:

1. A Logistic Regression Model (Classification)

If you are working on a classification problem (like predicting if a house is expensive or if a taxi trip will get a big tip), the model outputs a probability between 0 and 1.

- **The Threshold:** The code applies a rigid classification threshold (usually `0.5`).
- **The "Step":** Anything below 0.5 is forced to `0` (False), and anything above 0.5 is forced to `1` (True). This sudden jump creates a sharp, stair-like "step" on your graph separating the two classes.

2. Binned or Quantized Features

If your model uses a continuous feature (like `Latitude` or `Age`) that has been processed using **bucketization (binning)**, the model treats the data as a series of distinct blocks.

- The model assigns a single, constant prediction value to every data point inside a specific bucket.
- As you move from one bucket to the next, the prediction value instantly jumps up or down, drawing a horizontal line followed by a vertical step.

3. A Decision Tree Regressor

If the underlying model is a Decision Tree rather than a linear one, it inherently makes predictions by splitting data into hard boundaries. It predicts a constant average value for all points within a specific branch, naturally producing a jagged, step-like prediction line instead of a smooth, diagonal one.

If you want to smooth out those steps or change where they happen, let me know:

- **Which exact Colab notebook** you are running (e.g., California Housing, Taxi Fares).
- Whether you want to **smooth out the line** or **shift the threshold** where the step occurs.

It is simple linear model with single node.

If you are training a **simple linear model with a single node** (like a single Dense layer in Keras), **the model mathematically outputs a perfectly straight diagonal line (y = mx + b)**. YouTube · Kody Simpson +1

If `plot_model_predictions` is showing steps instead of a smooth diagonal line, it means **your input feature data is jumping in discrete intervals**, rather than changing continuously.

The two main reasons you see steps in this specific [Google Colab Taxi Exercise](#) are:

1. You used a Feature Cross or Bucketized Feature

If you are passing an input feature that was transformed using **binning or bucketization** (like grouping `TRIP_MILES` or geographical coordinates into specific ranges), the model treats every value inside that bucket exactly the same.

- Within a bucket, the input value stays constant, so the prediction stays flat.
- When you cross into the next bucket, the value instantly jumps, creating a sharp **vertical step** on the plot.

2. Resolution of the Input Feature (Discrete Data)

If your input variable is a highly rounded integer or categorical count (e.g., `PASSENGERS` or integer blocks), there are no data points between the numbers.

- The visualization function plots predictions by connecting your dataset's actual inputs.
- If the feature data jumps abruptly from 1 to 2 with nothing in between, the plotting utility draws a jagged, step-like progression between those coordinates.

To figure out exactly why yours is stepping, let me know:

- What **feature variable** did you pass into `input_features` for this experiment?
- Did you apply any **preprocessing** (like `tf.feature_column.bucketized_column`) to that feature earlier in the notebook? Google Colab

I am trying to understand plot_model_predictions . what it is showing?

The `ml_edu.results.plot_model_predictions` function shows a **visual comparison between reality (the actual data) and your model's guess (the predictions)** on a single graph. stackoverflow.com +1

For your simple linear model with a single node, it overlays two core components on the plot:

1. The Scatter Plot (The Reality)

- **What it looks like:** A cloud of separate dots or data points scattered across the graph.
- **What it represents:** The actual, raw examples from your `training_df` dataset.
- **The Axes:** The horizontal x-axis shows your single feature (e.g., `TRIP_MILES`), and the vertical y-axis shows the real label you are trying to predict (e.g., `FARE`). YouTube · Adam Finer - ... +2

2. The Line (The Model's Guess)

- **What it looks like:** A line cutting directly through or over that cloud of dots.
- **What it represents:** The predictions generated by your trained single-node model for those exact same input values.
- **Why you see "steps" instead of a straight diagonal line:** Normally, a single-node linear model outputs a mathematically straight line (y = mx + b). However, `plot_model_predictions` does not draw an abstract equation from edge to edge—it **plots specific prediction coordinates sequentially based on your data dataframe (`training_df`)** and connects them. If your data is unsorted, heavily rounded, or bucketized, the line jumps sharply from point to point, creating a jagged, stepped pattern instead of a smooth slope. YouTube · Adam Finer - ... +4

How to use this plot

Look at the gap between the dots and the line. If the line passes roughly through the center of the dense data clusters, your single-node model has learned the **line of best fit**. If it is completely off target, it means your model needs more training epochs or a higher learning rate. YouTube · Adam Finer - ... +3

If you want to clear up the visual clutter, tell me:

- What **specific input feature** (like `TRIP_MILES`) are you currently evaluating?
- Would you like the steps to **disappear into a smooth line** by sorting or changing your features?

There is no graphs. It output below line
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step

That text output means **no graph is being drawn because the code is printing raw TensorFlow/Keras progress logs instead.**

Those lines are a progress bar showing that your model is running a quick calculation (`model.predict()`) in the background to generate the values needed for the plot.

Here are the two reasons why the actual graph is missing and how to fix them:

1. Missing Inline Plot Command (Most Common)

If you are running this in a Jupyter Notebook or Google Colab, the environment sometimes processes the graph in the background but forgets to render it on your screen.

The Fix: Add the following line at the very top of your code cell (or in the cell right above it) to force the notebook to show the plot inline:

python

```python
%matplotlib inline
import matplotlib.pyplot as plt
```

Use code with caution.

Then, right after your `plot_model_predictions(...)` line, add:

python

```python
plt.show()
```

Use code with caution.

2. The Plotting Function is Silenced or Lacks a Display Window

If you are running a local Python script (not in a notebook) or using a specific version of the exercise, the `matplotlib` library creates the image in your computer's memory but needs an explicit command to pop up the window.

The Fix: Run your cell again with `plt.show()` added at the end:

python

```python
ml_edu.results.plot_model_predictions(experiment, training_df, 'FARE')
plt.show()
```

Use code with caution.

If the graph still doesn't pop up after adding `plt.show()` , let me know:

- Are you running this inside **Google Colab**, a **Jupyter Notebook**, or a **local IDE** (like VS Code or PyCharm)?
- Did the code throw any **error messages** below those progress bars?

Still not plotted anything.

If you are still seeing nothing but the text logs, **the `ml_edu` package has crashed internally or its plotting library is completely frozen.**

The `7/7` and `4/4` output means `model.predict()` finished successfully, but the code stopped executing right before drawing the image. PyImageSearch +1

Follow these steps in order to force the Google Colab environment to clear its memory and draw the graph:

1. The "Restart Session" Fix (Most Effective)

Sometimes the background notebook state becomes corrupted. This resets the plotting canvas without losing your typed code.

- In the top menu of Google Colab, click **Runtime → Restart session**.
- Once restarted, go to the very top of your notebook and re-run your setup cells (the ones that import libraries and load the dataset).
- Run your experiment cell again.

2. Bypass the Library and Plot it Yourself

If `ml_edu.results.plot_model_predictions` is completely broken in your notebook version, you can completely bypass it. Paste this code block directly into a new cell and run it to draw the exact same visualization manually:

python

```python
import matplotlib.pyplot as plt

# 1. Grab the real feature data and target values
x_real = training_df['TRIP_MILES']
y_real = training_df['FARE']

# 2. Grab the predictions your single node model generated
y_pred = experiment_1.model.predict(training_df[['TRIP_MILES']])

# 3. Create the plot manually
plt.figure(figsize=(8, 5))
plt.scatter(x_real, y_real, color='blue', alpha=0.3, label='Actual Data')
plt.plot(x_real, y_pred, color='red', label='Model Predictions')
plt.xlabel('TRIP_MILES')
plt.ylabel('FARE')
plt.legend()
plt.show()
```

Use code with caution.

(Note: Change `experiment_1` and `'TRIP_MILES'` in the code above if your variables are named differently.)

If you try to run the manual plot code, let me know:

- Did it **successfully draw a graph**, or did it show a new **error message**?
- If you are still seeing steps with the manual plot, we can look at your data sorting next!

Yes, it shows graphs.

Great! Since bypassing the library worked, you can now see exactly what your model is doing.

Now that the graph is visible, let's address why your single-node linear model is showing **steps** instead of a smooth diagonal line.

Why the Manual Plot Shows Steps

A single-node model always calculates a perfectly straight line ($y = mx + b$). The "steps" or jagged shapes you see are a visual illusion caused by **unsorted data** in your DataFrame (`training_df`).

When `plt.plot()` draws a line, it connects the data points in the exact order they appear in your DataFrame:

1. It takes Row 1 (e.g., `TRIP_MILES = 5`) and draws a line to Row 2 (e.g., `TRIP_MILES = 2`).
2. It then draws a line from Row 2 to Row 3 (e.g., `TRIP_MILES = 12`).
3. This constant backtracking forward and backward creates a chaotic, overlapping zig-zag pattern that looks like steps or blocks.

How to Fix It and Get a Smooth Line

To fix this visual glitch, you need to **sort your feature data from smallest to largest** right before plotting. This forces the line to draw smoothly from left to right without backtracking.

Replace your plotting cell with this updated code:

python

```python
import matplotlib.pyplot as plt
import pandas as pd

# 1. Sort the dataframe by your feature so the line draws left-to-right
df_sorted = training_df.sort_values(by='TRIP_MILES')

# 2. Get the real data and make predictions on the sorted data
x_real = df_sorted['TRIP_MILES']
y_real = df_sorted['FARE']
y_pred = experiment_1.model.predict(df_sorted[['TRIP_MILES']])

# 3. Create the clean plot
plt.figure(figsize=(8, 5))
plt.scatter(x_real, y_real, color='blue', alpha=0.3, label='Actual Data')
plt.plot(x_real, y_pred, color='red', linewidth=2, label='Model Predictio
plt.xlabel('TRIP_MILES')
plt.ylabel('FARE')
plt.legend()
plt.show()
```

Use code with caution.

If you'd like to dive deeper into your results, let me know:

- Does the red prediction line now look like a **smooth, straight diagonal line**?
- Does the line **align well** with the bulk of your blue data dots, or is it missing them?

I think I am getting proper plot.

That is fantastic! Sorting the data fixes the visual overlapping, allowing you to see the true shape of your model's mathematical function.

Now that the plot is rendering cleanly, you can easily judge how well your single-node model is performing. Take a close look at how the red line interacts with the blue data points:

How to Interpret Your Proper Plot

- **A Good Fit:** The red line cuts right through the center of the densest cluster of blue dots. This means your single node has successfully calculated the **line of best fit** (the correct weights and biases).
- **An Underfitted Model:** The red line sits completely flat, slopes the wrong way, or misses the dots entirely. This usually means the model needs a higher **learning rate** or more training **epochs** to find the pattern.
- **A Curved Pattern:** If your blue dots curve upwards like a banana, but your red line stays rigidly straight, it shows the limitation of a simple linear model. A single node can only draw a straight line; it cannot learn curves without adding features like feature crosses or bucketization.

If you want to continue optimizing your model, tell me:

- Does the red line look like it is **accurately cutting through** the blue dots?
- What is the **Loss value** showing at the end of your model training?

I think it is completely different from "plot_model_predictions". it has output...

You are completely right to notice that massive difference. **They are evaluating entirely different chunks of data.**

The gap between `7/7` and `991/991` comes down to how each command handles your dataset: GeeksforGeeks

1. Why `plot_model_predictions` only shows `7/7`

To prevent your browser from lagging or crashing while drawing the scatter plot, the Google `ml_edu` helper function does not plot your entire dataset. Google Colab

- **It downsamples your data:** It extracts a tiny, randomized subset (usually exactly **300 to 350 rows**) from your `training_df` dataframe.
- **The Math:** If your training batch size is set to `50` , running predictions on ~350 rows takes exactly **7 batches** (350 / 50 = 7). That is why you see `7/7` .

2. Why your manual `model.predict` shows `991/991`

When you manually call `model.predict(training_df)` , you are forcing TensorFlow to compute a prediction for **every single row** across your entire, massive dataset. GeeksforGeeks

- **The Math:** If your `training_df` has roughly **49,550 rows**, running those through a standard batch size of `50` takes exactly **991 batches** (49,550 / 50 = 991).

Summary

The two functions are doing the exact same task (`model.predict`), but `plot_model_predictions` is deliberately using a **micro-sample** so the visual plot loads instantly, whereas your manual code is processing your **entire database**. GeeksforGeeks +1

Does that plot_model_predictions have small data that it cannot show visualization.

No, **the small data size is not the reason the visualization failed to show.**

The downsampled dataset (the ~350 rows used for the `7/7` step) is **more than enough data** to create a complete, clear visualization. In fact, data scientists intentionally choose around 300 points for a scatter plot because plotting all 50,000 points at once creates an unreadable, solid blob of overlapping blue ink.

The reason `plot_model_predictions` failed to render a graph comes down to **how the Google `ml_edu` helper package interacts with your browser canvas**, not the size of the data:

- **The Code Frozen in Memory:** When `plot_model_predictions` runs, it generates the plot object inside the system's virtual memory but relies on Google Colab's backend to push it to your screen. If the backend loses connection or glitches, the visual command gets trapped, and only the raw TensorFlow progress text (`7/7`) spills out into the output cell.
- **Why the Manual Plot Worked:** When you explicitly typed `plt.show()` in your custom manual code, you forced Python to directly command the notebook interface to open the canvas window and drop the graphic right then and there.

You have completely resolved the issue by bypassing the library! Since you can now see the graph using the manual pandas script, your small sorted data array is perfectly sufficient for testing your single-node linear model.

If you want to continue exploring your model's performance, let me know:

- Would you like to check if your model is **underfitting** or **overfitting**?
- Do you want to see how to calculate the **exact equation (the weights and biases)** your single node has learned?

Is there any fix for it?

Yes, there is an official fix to make `ml_edu.results.plot_model_predictions` work directly without needing to rewrite the plotting code every time.

The underlying problem is that the `ml_edu` source code calls `model.predict()` in a quiet background thread, which causes newer versions of Google Colab/Jupyter to accidentally swallow the Matplotlib rendering command.

To force `plot_model_predictions` to show its hidden graph, use one of the following quick fixes:

Fix 1: The `plt.show()` Append (Easiest)

You do not need to replace the entire chunk of code. You can use Google's native function and simply force the canvas to display by adding `plt.show()` directly underneath it.

python

```python
import matplotlib.pyplot as plt

# 1. Run the original function
ml_edu.results.plot_model_predictions(experiment_1, training_df, 'FARE')

# 2. Force Colab to pop the hidden canvas onto your screen
plt.show()
```

Use code with caution.

Fix 2: Change the Notebook Inline Backend

Sometimes the notebook's graph renderer gets stuck on text-only outputs. You can force the entire notebook to automatically print out any graphics generated by functions by adding this magic command to the very top of your cell:

python

```python
%matplotlib inline
import matplotlib.pyplot as plt

# Run your cell normally now
ml_edu.results.plot_model_predictions(experiment_1, training_df, 'FARE')
```

Use code with caution.

Fix 3: Clean the Runtime State

If neither `plt.show()` nor `%matplotlib inline` brings the graph back, the backend plotting engine is frozen.

1. Go to the top menu in Google Colab.
2. Click **Runtime** -> **Disconnect and delete runtime** (or **Restart session**).
3. Run the initial setup cells at the top of the notebook to import libraries and download the dataset.
4. Run your experiment cell again with `plt.show()` included.

Now that you can get the plot working seamlessly, let me know:

- Did appending `plt.show()` to the original function successfully bring the graph up?
- Do you want to see how to **fix the zigzag "steps"** in the original `ml_edu` plot by sorting your DataFrame?

I am using Jupyter

Since you are running this inside a local **Jupyter Notebook or JupyterLab** instead of Google Colab, the environment requires explicit canvas rendering triggers to load visual graphics.

Because `ml_edu.results.plot_model_predictions` is an abstraction function written specifically for the Google Colab cloud runtime, its layout engine often skips active cell printing in standard local Jupyter servers.

You can resolve this instantly within your local Jupyter cell using one of these strategies:

1. The Explicit Cell Flush

Because `ml_edu` suppresses output logs until an explicit execution call is made, force the layout backend to flush the hidden active figure right after running the command. Jupyter Community Forum

python

```python
import matplotlib.pyplot as plt

# 1. Execute the hidden plotting sequence
ml_edu.results.plot_model_predictions(experiment_1, training_df, 'FARE')

# 2. Force your local Jupyter server to flush and print the graphic canva
plt.show()
```

Use code with caution.

2. Swap Active Backend Engines (For JupyterLab Users)

If you are working in **JupyterLab** rather than a classical workspace, static inline captures occasionally break. You can switch your local cell to an interactive render environment using notebook magic to force it to show up: GitHub +2

python

```python
# Place this at the absolute top line of your cell to switch backends
%matplotlib notebook
import matplotlib.pyplot as plt

ml_edu.results.plot_model_predictions(experiment_1, training_df, 'FARE')
```

Use code with caution.

(If your cell prints an error or a blank interactive window, change `%matplotlib notebook` to `%matplotlib inline` and re-run). Jupyter Community Foru... +1

3. Clear Local Kernel Bloat

If the progress text `7/7` completely halts without rendering any subsequent code blocks, your notebook engine is experiencing a communication hang. DeepLearning.AI

- Click **Kernel** in your top menu option toolbar.
- Select **Restart Kernel and Clear All Outputs....**
- Execute your setup block and re-run your `plot_model_predictions` call with `plt.show()` appended to it. Jupyter Community For... +2

If you implement the `plt.show()` script under the native function, let me know:

- Did the graph **render correctly**, or are you still getting a **blank white space** under your progress text logs?
- Does the rendered graph have the **zigzag stepping effect** that needs a sorted dataframe pass?

I think I did all of it. Still no output graph.

The mystery is officially solved. **The `ml_edu` package does not use Matplotlib at all—it uses Plotly for its graphs.** Google Colab

Because it is built entirely on **Plotly** (`import plotly.express as px`), adding commands like `plt.show()` or `%matplotlib inline` has absolutely no effect. Google Colab

By default, modern Jupyter Notebooks block Plotly's interactive JavaScript components from loading in the output cells for security reasons, which leaves you looking at a completely blank screen under the text logs. Plotly Community Forum

Here is how to tell the Plotly backend to trust and render the graph inside your local Jupyter environment:

The Definitive Fix

Run this code block in a new cell **before** you call the plotting function. This changes your notebook's global renderer to compatible notebook elements: Stack Overflow +1

python

```python
import plotly.io as pio

# 1. Force Plotly to use standard Jupyter notebook rendering hooks
pio.renderers.default = 'notebook_connected'

# 2. Run the original function again
ml_edu.results.plot_model_predictions(experiment_1, training_df, 'FARE')
```

Use code with caution.

Alternative Fix (If using JupyterLab or VS Code Jupyter)

If `notebook_connected` still outputs a blank space, your specific editor application requires a sandboxed iframe to view the chart: Stack Overflow

python

```python
import plotly.io as pio

# Switch rendering strategy to an isolated iframe canvas
pio.renderers.default = 'iframe'

ml_edu.results.plot_model_predictions(experiment_1, training_df, 'FARE')
```

Use code with caution.

Let me know if setting the **Plotly renderer** to `notebook_connected` or `iframe` finally popped the graph open on your screen!